

Maior Subsequência Comum

Descrição da solução

O programa recebe duas sequências de tickers de ações da B3, representadas por dois vetores de Strings, com tamanhos M e N. O objetivo é encontrar a maior subsequência que aparece nas duas sequências, mantendo a mesma ordem em que os tickers aparecem em cada uma delas.

Para uma sequência ser considerada válida, os tickers precisam aparecer nas duas corretoras e a posição de cada novo ticker precisa ser posterior à posição do ticker anterior nas duas sequências.

A solução começa lendo os valores M e N e criando os dois vetores. Em seguida, os tickers são armazenados nos vetores das respectivas corretoras.

Para encontrar os elementos em comum, são utilizados dois laços de repetição. O primeiro percorre a primeira sequência e o segundo percorre a segunda sequência. Quando dois tickers são iguais, suas posições nas duas sequências são identificadas.

Depois disso, é utilizada uma busca recursiva para continuar procurando outros tickers que possam fazer parte da subsequência. A busca recebe como referência as últimas posições utilizadas nas duas sequências. Dessa forma, um novo ticker só pode ser escolhido se estiver depois das posições anteriores nas duas corretoras.

A sequência que está sendo testada é armazenada em uma lista chamada de sequência atual. Quando um ticker válido é encontrado, ele é adicionado à sequência. O algoritmo então continua a busca a partir das novas posições.

Após explorar uma possibilidade, o último ticker escolhido é removido da sequência atual. Dessa forma, o programa consegue voltar para uma situação anterior e testar outras possibilidades. Esse processo é utilizado para realizar o backtracking.

Durante a execução, também é mantida uma lista contendo a melhor subsequência encontrada até aquele momento. Quando uma sequência maior é encontrada, ela passa a ser armazenada como a melhor solução.

Ao final da execução, o programa apresenta o tamanho da maior subsequência encontrada e os tickers que fazem parte dela.

Modelagem

A primeira sequência é representada pelo vetor corretoraA e a segunda sequência pelo vetor corretoraB. Cada posição dos vetores representa um ticker.

A busca utiliza duas posições para controlar a ordem dos elementos. A última posição utilizada na primeira sequência e a última posição utilizada na segunda sequência são passadas para a próxima etapa da busca.

A sequência atual representa os tickers escolhidos durante um determinado caminho da busca. A melhor sequência representa a maior subsequência encontrada entre todos os caminhos analisados.

A busca começa com as posições anteriores definidas como menos um, pois inicialmente nenhum ticker foi escolhido. Assim, o primeiro elemento pode ser procurado desde a posição zero.

Complexidade de tempo (O)

A solução utiliza dois laços para comparar os elementos das duas sequências. Entretanto, quando encontra elementos iguais, realiza chamadas recursivas e explora diferentes possibilidades de subsequências.

No pior caso, a quantidade de possibilidades cresce exponencialmente em relação ao tamanho da menor sequência. Considerando essa característica, a complexidade de tempo da solução é $O(M \text{ vezes } N \text{ vezes } 2^{\text{elevado ao mínimo entre } M \text{ e } N})$.

O fator $M \text{ vezes } N$ está relacionado às comparações entre os elementos das duas sequências, enquanto o fator exponencial está relacionado à exploração das diferentes possibilidades durante o processo de backtracking.

Por esse motivo, o tempo de execução pode aumentar bastante conforme os valores de M e N aumentam, principalmente quando existem muitos elementos em comum entre as duas sequências.

Complexidade de espaço

Os dois vetores utilizados para armazenar as entradas ocupam espaço proporcional a M mais N .

Além disso, a sequência atual pode armazenar uma quantidade de elementos proporcional ao tamanho da menor sequência, assim como a profundidade da chamada recursiva.

Considerando os vetores de entrada e as estruturas utilizadas durante a execução, a complexidade de espaço é $O(M \text{ mais } N)$.

Análise experimental

Para analisar o comportamento da solução, podem ser realizados testes aumentando gradualmente os valores de M e N . Os tempos de execução devem ser medidos durante os

testes e registrados para comparar o comportamento do programa conforme o tamanho das entradas aumenta.

É esperado que o tempo de execução aumente significativamente em entradas maiores, principalmente quando existem muitos tickers em comum, pois nesse caso o algoritmo precisa explorar uma quantidade maior de possibilidades.

Análise experimental

Para analisar o comportamento do programa, podem ser realizados testes aumentando gradualmente os valores de M e N

Um exemplo de sequência de testes é:

M = N	$2N2^N$	Crescimento relativo
5	32	32
10	1.024	1 mil
15	32.768	33 mil
20	1.048.576	1 milhão
25	33.554.432	33 milhões
30	1.073.741.824	1 bilhão
35	34.359.738.368	34 bilhões
50	1.099.511.627.776	1,1 trilhão

55	1.125.899.906.842.624	1,1 quatrilhão
60	1.152.921.504.606.846.976	~1,15 quintilhão

A expectativa é que o tempo aumente de forma significativa conforme o tamanho das entradas aumenta, principalmente quando existem muitos tickers em comum, pois isso gera mais possibilidades para a busca explorar.

Uso de Inteligência Artificial

Foi utilizada Inteligência Artificial como ferramenta de apoio durante o desenvolvimento do trabalho. A ferramenta foi utilizada principalmente para auxiliar na compreensão de conceitos de programação em Java, como vetores, comparação de Strings, ArrayList, recursão e backtracking, além de auxiliar na compreensão da análise de complexidade.